Nama: Ariel Ardani Aris Putra

NIM: 254107020129

No. Absen: 04

Kelas: TI 1G

5.2 Menghitung Nilai Faktorial dengan Algoritma Brute Force dan Divide and Conquer

5.2.1. Langkah-langkah Percobaan

Class Faktorial

```
package BruteForceDivideConquer.minggu5;


public class Faktorial {
    int faktorialBF(int n) {
    int fakto = 1;
    for (int i = 1; i <= n; i++) {
        fakto = fakto * i;
    }
    return fakto;
}
int faktorialDC(int n) {
    if (n == 1) {
        return 1;
    } else {
        int fakto = n * faktorialDC(n - 1);
        return fakto;
    }
}
}
```

Class mainFaktorial

```
package BruteForceDivideConquer.minggu5;
```

```
import java.util.Scanner;

public class mainFaktorial {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Masukkan nilai: ");
        int nilai = input.nextInt();
        Faktorial fk = new Faktorial();

        System.out.println("Nilai faktorial " + nilai +
                " menggunakan BF: " + fk.faktorialBF(nilai));

        System.out.println("Nilai faktorial " + nilai +
                " menggunakan DC: " + fk.faktorialDC(nilai));
    }
}
```

5.2.2. Verifikasi Hasil Percobaan

```
Masukkan nilai: 12
Nilai faktorial 12 menggunakan BF: 479001600
Nilai faktorial 12 menggunakan DC: 479001600
PS E:\JavaGit\OperatingSystem26>
```

5.2.3. Pertanyaan

1. Pada base line Algoritma Divide Conquer untuk melakukan pencarian nilai faktorial, jelaskan perbedaan bagian kode pada penggunaan if dan else!
2. Apakah memungkinkan perulangan pada method `faktorialBF()` diubah selain menggunakan for? Buktikan!
3. Jelaskan perbedaan antara `fakto *= i;` dan int `fakto = n * faktorialDC(n-1);` !
4. Buat Kesimpulan tentang perbedaan cara kerja method `faktorialBF()` dan `faktorialDC()`!

jawab!

1. if pada pemilihan tersebut menjadi base case.

   else pada pemilihan tersebut menjadi fungsi rekursif dimana n akan dikalikan dengan fungsi itu sendiri dengan parameter n-1.

2. bisa di ubah menggunakan while

   ```
   int i=;
     while (i <= n) {
         fakto = fakto * i;
         i++;
      }
   ```

3. - fakto *= i; apabila di artikan akan menjadi fakto = fakto * i. program ini akan mengalikan fakto dengan i yang dimana nilai i nya akan di inkremen hingga kondisi i=n dan berhenti apabila i>n.

   - fakto = n * faktorialDC(n - 1); program ini mengalikan n dengan hasil dari fungsi faktorialDC(n-1); dengan ini n akan dikalikan dengan n-1 terus menerus hingga nilai n=1 (n*(n-1)*(n-1)...) pada setiap iterasi.

4. fakrotialBF() menggunakan iterasi dimana ini straight mengalikan nilai dari 1 hingga n menggunakan looping.

   faktorialDC() menggunakan fungsi rekursif dimana ini akan mengalikan nilai n dengan (n-1)*(n-1) secara terus menerus hingga nilai n=1

5.3 Menghitung Hasil Pangkat dengan Algoritma Brute Force dan Divide and Conquer

5.3.1. Langkah-langkah Percobaan

Class Pangkat

```
package BruteForceDivideConquer.minggu5;


public class Pangkat {
   int nilai, pangkat;


   Pangkat(int n, int p) {
      nilai = n;
      pangkat = p;
   }
```

```
    int pangkatBF(int a, int n) {
        int hasil = 1;
        for (int i = 0; i < n; i++) {
            hasil = hasil * a;
        }
        return hasil;
    }

    int pangkatDC(int a, int n) {
        if (n == 1) {
            return a;
        } else {
            if (n % 2 == 1) {
                return (pangkatDC(a, n / 2) * pangkatDC(a, n / 2) * a);
            } else {
                return (pangkatDC(a, n / 2) * pangkatDC(a, n / 2));
            }
        }
    }
}
```

Class mainPangkat

```
package BruteForceDivideConquer.minggu5;

import java.util.Scanner;

public class mainPangkat {
    public static void main(String[] args) {
```

```
        Scanner input = new Scanner(System.in);
        System.out.print("Masukkan jumlah elemen: ");
        int elemen = input.nextInt();
        Pangkat[] png = new Pangkat[elemen];
        for (int i = 0; i < elemen; i++) {
            System.out.print("Masukan nilai basis elemen ke-" + (i + 1) + ": ");
            int basis = input.nextInt();
            System.out.print("Masukan nilai pangkat elemen ke-" + (i + 1) + ": ");
            int pangkat = input.nextInt();
            png[i] = new Pangkat(basis, pangkat);
        }
        System.out.println("HASIL PANGKAT BRUTE FORCE:");
        for (Pangkat p : png) {
            System.out.println(p.nilai + "^" + p.pangkat + ": " + p.pangkatBF(p.nilai,
p.pangkat));
        }

        System.out.println("HASIL PANGKAT DIVIDE AND CONQUER:");
        for (Pangkat p : png) {
            System.out.println(p.nilai + "^" + p.pangkat + ": " + p.pangkatDC(p.nilai,
p.pangkat));
        }
    }
}
```

5.3.2. Verifikasi Hasil Percobaan

```
Masukkan jumlah elemen: 3
Masukan nilai basis elemen ke-1: 1
Masukan nilai pangkat elemen ke-1: 2
Masukan nilai basis elemen ke-2: 3
Masukan nilai pangkat elemen ke-2: 4
Masukan nilai basis elemen ke-3: 5
Masukan nilai pangkat elemen ke-3: 6
HASIL PANGKAT BRUTE FORCE:
1^2: 1
3^4: 81
5^6: 15625
HASIL PANGKAT DIVIDE AND CONQUER:
1^2: 1
3^4: 81
5^6: 15625
```

5.3.3. Pertanyaan

1. Jelaskan mengenai perbedaan 2 method yang dibuat yaitu `pangkatBF()` dan `pangkatDC()`!
2. Apakah tahap *combine* sudah termasuk dalam kode tersebut?Tunjukkan!
3. Pada method `pangkatBF()` terdapat parameter untuk melewatkan nilai yang akan dipangkatkan dan pangkat berapa, padahal di sisi lain di class `Pangkat` telah ada atribut `nilai` dan `pangkat`, apakah menurut Anda method tersebut tetap relevan untuk memiliki parameter? Apakah bisa jika method tersebut dibuat dengan tanpa parameter? Jika bisa, seperti apa method `pangkatBF()` yang tanpa parameter?
4. Tarik tentang cara kerja method `pangkatBF()` dan `pangkatDC()`!

jawab!

1. pangkatBF() menggunakan method brute force dimana ini akan straight ke hasil dengan cara melakukan loop sebanyak n lalu pada setiap iterasi menjalankan hasil = hasil *a. dimana ini akan membuat hasil dikali a sebanyak n

   PangkatDC() menggunakan method devide and conquer dimana ini akan memecah pangkat menjadi beberapa bagian semisal 2^6 program akan menjalankan 2^3 * 2^3 lalu pada 2^3 akan di pecah lagi menjadi 2^1*2^1*2 lalu hasil dari pemecahan tersebut akan di combine dengan cara dikalikan.

2. sudah ada pada method PangkatDC() menggunakan method devide and conquer dimana ini akan memecah pangkat menjadi beberapa bagian semisal 2^6 program akan menjalankan 2^3 * 2^3 lalu pada 2^3 akan di pecah lagi menjadi 2^1*2^1*2 lalu hasil dari pemecahan tersebut akan di combine dengan cara dikalikan.
3. kurang relevan, karena atribut menjadi tidak terpakai.

kode program tanpa parameter

```
int pangkatBF() {
    int hasil = 1;
    for (int i = 0; i < pangkat; i++) {
        hasil = hasil * nilai;
    }
    return hasil;
}
```

4. pangkatBF() straight ke hasil dengan cara melakukan loop sebanyak n lalu pada setiap iterasi menjalankan hasil = hasil *a. dimana ini akan membuat hasil dikali a sebanyak n.

   pangkatDC() memecah pangkat menjadi beberapa bagian semisal 2^6 program akan menjalankan 2^3 * 2^3 lalu pada 2^3 akan di pecah lagi menjadi 2^1*2^1*2 lalu hasil dari pemecahan tersebut akan di combine dengan cara dikalikan.

5.4 Menghitung Sum Array dengan Algoritma Brute Force dan Divide and Conquer

5.4.1. Langkah-langkah Percobaan

Class Sum

```
package BruteForceDivideConquer.minggu5;


public class Sum {
    double keuntungan[];


    Sum(int el) {
        keuntungan = new double[el];
    }


    double totalBF() {
        double total = 0;
        for (int i = 0; i < keuntungan.length; i++) {
```

```
            total = total + keuntungan[i];
        }
        return total;
    }

    double totalDC(double arr[], int l, int r) {
        if (l == r) {
            return arr[l];
        }

        int mid = (l + r) / 2;
        double lsum = totalDC(arr, l, mid);
        double rsum = totalDC(arr, mid + 1, r);
        return lsum + rsum;
    }
}
```

Class mainSum

```
package BruteForceDivideConquer.minggu5;

import java.util.Scanner;

public class mainSum {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Masukkan jumlah elemen: ");
        int elemen = input.nextInt();
        Sum sm = new Sum(elemen);
        for (int i = 0; i < elemen; i++) {
```

```
            System.out.print("Masukkan keuntungan ke-" + (i + 1) + ": ");
            sm.keuntungan[i] = input.nextDouble();
        }
        System.out.println("Total keuntungan menggunakan Bruteforce: " +
sm.totalBF());
        System.out.println(
                "Total keuntungan menggunakan Divide and Conquer: " +
sm.totalDC(sm.keuntungan, 0, elemen - 1));
    }

}
```

5.4.2. Verifikasi Hasil Percobaan

```
Masukkan jumlah elemen: 5
Masukkan keuntungan ke-1: 1
Masukkan keuntungan ke-2: 2
Masukkan keuntungan ke-3: 3
Masukkan keuntungan ke-4: 4
Masukkan keuntungan ke-5: 5
Total keuntungan menggunakan Bruteforce: 15.0
Total keuntungan menggunakan Divide and Conquer: 15.0
PS E:\JavaGit\PraktikumAlgoritmaStrukturData>
```

5.4.3. Pertanyaan

1. Kenapa dibutuhkan variable `mid` pada method `TotalDC()`?
2. Untuk apakah statement di bawah ini dilakukan dalam `TotalDC()`?

```
double lsum = totalDC(arr, l, mid);
double rsum = totalDC(arr, mid+1, r);
```

3. Kenapa diperlukan penjumlahan hasil lsum dan rsum seperti di bawah ini?

```
return lsum+rsum;
```

4. Apakah base case dari `totalDC()`?
5. Tarik Kesimpulan tentang cara kerja `totalDC()`

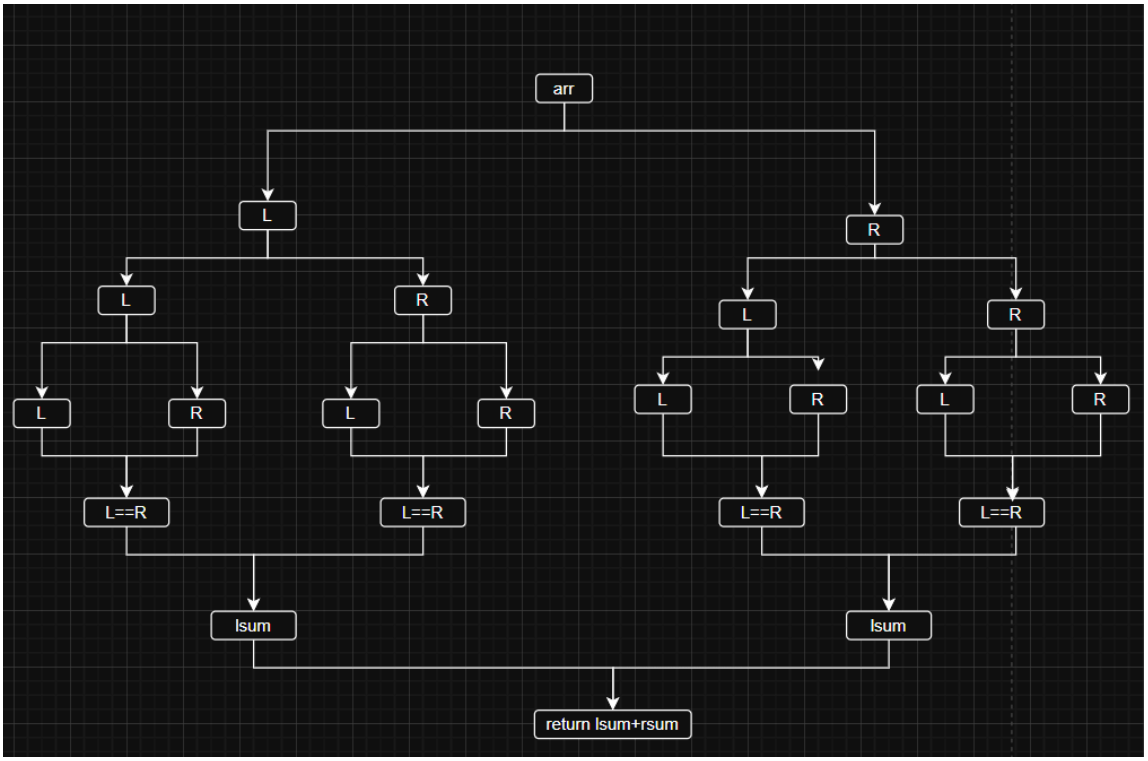
1. variable mid dipakai sebagai parameter pada fungsi rekursif

2. melakukan perulangan rekursif dimana lsum dihasilkan dari penjumlahan nilai array arr[l sampai mid] dan rsum[] dihasilkan dari penjumlaham nilai arr[mid sampai r].
3. penjumlahan tersebut berfungsi untuk mengcombine hasil dari lsum dan rsum.
4. base case nya

```
    if (l == r) {

        return arr[l];

    }
```

5. cara kerja totalDC() adalah dengan memecah data menjadi bagian kanan dan kiri lalu dilakukan perulangan rekursif untuk mencari total dari bagian kanan dan kiri dimana nanti bagian kanan dan kiri akan di pecah lagi menjadi bagian kanan dan kiri hingga tidak bisa di pecah.

   apabila di ilustrasikan